

SportCourt - Ứng dụng quản lý đặt và cho thuê sân thể thao

(bóng đá • cầu lông • pickleball)

Tài liệu kế hoạch triển khai và best practices (Java/Spring Boot)

1. Mục tiêu dự án và định hướng sản phẩm

Xây dựng ứng dụng quản lý sân thể thao theo mô hình SaaS, giúp chủ sân vận hành hiệu quả và khách đặt sân nhanh: minh bạch lịch trống theo thời gian thực, tránh chồng chéo lịch, tự động tính tiền theo khung giờ/loại sân, bán dịch vụ kèm theo, và báo cáo doanh thu/lợi nhuận.

1.1 Mục tiêu nghiệp vụ

- Đặt sân trực tuyến 24/7: khách tự xem lịch trống và đặt sân online, giảm cuộc gọi/nhắn tin thủ công.
- Tránh trùng lịch/chồng chéo: quản lý lịch đặt theo ngày/tuần/tháng; hỗ trợ đặt vắng lai, cố định theo tháng và linh động (khách chưa biết giờ kết thúc).
- Tự động tính tiền thuê sân: hệ thống tính tổng tiền theo giờ/khung giờ, giúp giảm thao tác và sai sót.
- Tăng doanh thu: bán dịch vụ/sản phẩm đi kèm (nước, bóng, vợt, thuê dụng cụ...), hỗ trợ tối ưu khung giờ thấp điểm bằng sự kiện/hoạt động cộng đồng.
- Báo cáo vận hành: doanh thu theo ngày/tháng/năm, phân tách vắng lai vs cố định, thống kê chi tiết để ra quyết định.

1.2 Mục tiêu kỹ thuật

- Modular monolith (Spring Boot) với ranh giới module rõ ràng: venue, booking, pricing, payment, sales, reporting...
- Dữ liệu nhất quán: transaction, optimistic locking, constraint chống overlap.
- Tự động hóa: nhắc lịch, xác nhận, xử lý quá hạn/no-show (scheduler + notification worker).
- Quan sát hệ thống: correlationId theo booking, metrics (tỷ lệ lấp đầy, số booking/hủy/đổi, lỗi payment).
- Tài liệu API rõ ràng (OpenAPI/Swagger), dữ liệu mẫu và chạy nhanh bằng Docker Compose.

1.3 Tích hợp Chatbot AI (Customer & Owner Assistant)

Chatbot AI là trợ lý cho khách hàng và chủ sân, ưu tiên trả lời dựa trên dữ liệu thật trong hệ thống (lịch trống, bảng giá, booking, chính sách, báo cáo).

Phạm vi gợi ý:

- Khách hỏi: “Tối nay 19:00–21:00 còn sân cầu lông nào trống?” → bot trả danh sách sân trống + gợi ý slot gần nhất.
- Khách hỏi: “Giá pickleball cuối tuần?” → bot trả bảng giá theo khung giờ + phí/đặt cọc + chính sách hủy.
- Khách hỏi: “Đặt sân bóng đá 7 người 90 phút” → bot tạo booking draft (giữ chỗ X phút) và gửi link xác nhận.
- Chủ sân hỏi: “Sân nào đang trống nhiều?” → bot trả occupancy theo giờ/sân và đề xuất khuyến mãi.
- Chủ sân hỏi: “Doanh thu tuần này theo chi nhánh?” → bot trả báo cáo tổng hợp.

Best practice cho chatbot:

- Permission-aware (RBAC): OWNER/STAFF thấy toàn bộ theo chi nhánh; CUSTOMER chỉ thấy booking của mình.
- Bot không thực hiện hành động nhạy cảm trực tiếp (hủy/đổi/hoàn tiền): chỉ tạo đề xuất, người dùng xác nhận qua flow chuẩn.
- Log tối thiểu (intent, latency) và mask dữ liệu nhạy cảm khi lưu.
- Nếu bot không chắc, trả hướng dẫn hoặc đề nghị chọn lại khung giờ/sân.

2. Phạm vi triển khai

2.1 Tính năng cho Chủ sân / Nhân viên

- Quản lý chi nhánh (tùy chọn) và sân: loại sân (FOOTBALL/BADMINTON/PICKLEBALL), số sân, cấu hình slot 30/60 phút.
- Xem trạng thái sân theo 2 chế độ: theo giờ và theo sân (timeline vs court view).
- Quản lý lịch đặt: vãng lai theo ngày, cố định theo tháng, linh động (open-ended).
- Bảng giá linh động: theo khung giờ, ngày thường/cuối tuần, nhóm khách (member/vip).
- Bán dịch vụ/sản phẩm: đồ uống, thuê dụng cụ, phụ thu đèn/giờ đêm, trọng tài (đính kèm booking).
- Báo cáo: doanh thu, tỷ lệ lấp đầy theo khung giờ, top sân, top dịch vụ bán kèm.

2.2 Tính năng cho Khách hàng

- Đăng ký/đăng nhập, quản lý hồ sơ.
- Xem lịch trống theo ngày, đặt sân online, nhận thông báo xác nhận.
- Hủy/đổi lịch theo policy (giới hạn: chỉ hủy trước X giờ).
- Xem lịch sử đặt sân và hóa đơn/biên nhận.

2.3 Payment & Notification (tối giản)

- Thanh toán mock hoặc tích hợp sau; hỗ trợ đặt cọc và trả phần còn lại tại sân.
- Notification mock (email/SMS/Zalo): xác nhận, nhắc lịch, nhắc thanh toán.

3. Kiến trúc

Hệ thống được thiết kế theo kiến trúc microservices ở mức cơ bản, tập trung tách riêng các phần để mở rộng về sau (thanh toán, thông báo, AI) khỏi lõi booking. Điều này giúp tích hợp thêm cổng thanh toán/đa kênh notification mà không ảnh hưởng nhiều tới dịch vụ đặt sân.

3.1 Danh sách services (cơ bản nhưng đúng chuẩn)

- api-gateway: entry point duy nhất cho frontend/mobile; route tới các service; có rate limit và CORS
- auth-service (tối giản): phát hành JWT (hoặc dùng core-service phát JWT); các service verify JWT bằng public key
- core-service: venue/court, availability, booking, pricing, sales add-ons; state machine booking và chống overlap
- payment-service: payment intent, đặt cọc, webhook callback, idempotency, receipts; publish event kết quả thanh toán
- notification-service: email/SMS/Zalo (ban đầu mock), template, retry, lưu lịch sử gửi
- chatbot-service (tùy chọn): xử lý hội thoại, gọi LLM, permission-aware retrieval; tạo booking draft/tra cứu thông tin
- reporting-service (tùy chọn): read model/dashboard; tính occupancy, revenue, best-hours (có thể dựng sau)

3.2 Giao tiếp giữa services

- Synchronous (REST): gateway → services; core-service gọi payment-service để tạo payment intent (nếu cần).
- Asynchronous (event-driven): core-service và payment-service publish events; notification-service consume để gửi thông báo.
- Khuyến nghị dùng Outbox pattern để publish event an toàn (tránh mất event khi DB commit thành công nhưng publish thất bại).

Ví dụ events:

- BOOKING_CREATED, BOOKING_CONFIRMED, BOOKING_CANCELED
- PAYMENT_INTENT_CREATED, PAYMENT_SUCCEEDED, PAYMENT_FAILED
- NOTIFICATION_SENT, NOTIFICATION_FAILED

3.3 Data & Consistency (best practice)

- Database per service: core-db, payment-db, notification-db (không share schema).
- Không dùng distributed transaction; dùng saga-lite theo trạng thái: CONFIRMED_PENDING_PAYMENT → CONFIRMED/FAILED_TIMEOUT.

- Availability/booking conflict: enforce ở core-service bằng DB constraint/locking; cache lịch trống ở Redis để đọc nhanh.

3.4 Hạ tầng và triển khai

- Message broker: RabbitMQ (dễ setup) hoặc Kafka (nếu muốn scale lớn).
- Redis: cache availability, rate limit, lock nhẹ.
- Observability: structured logging + correlationId/tracelId; OpenTelemetry + Jaeger (tùy chọn).
- Local dev: Docker Compose chạy toàn bộ services + DB + broker + redis.

3.5 Công nghệ sử dụng

Backend

- Java 21 (hoặc 17) + Spring Boot 3
- Spring Web (REST), Spring Security (JWT/RBAC)
- Spring Data JPA + Hibernate; Flyway migration
- Spring Cloud Gateway (API Gateway)
- OpenFeign hoặc WebClient (service-to-service calls)
- Resilience4j (timeout/retry/circuit breaker)
- Messaging: RabbitMQ (Spring AMQP) hoặc Kafka (Spring for Apache Kafka)
- Outbox publisher (table outbox + worker) để publish event an toàn
- Database: PostgreSQL (mỗi service 1 DB), Redis (cache/rate limit)
- OpenAPI/Swagger (springdoc)
- Test: JUnit 5, Mockito, Testcontainers (Postgres/Redis/Broker)
- Observability: Actuator, log JSON, OpenTelemetry (optional)
- Build/DevOps: Maven/Gradle, Docker, Docker Compose, GitHub Actions CI

Frontend

- React + TypeScript
- UI: Tailwind CSS (hoặc MUI)
- Routing: React Router
- Data fetching: TanStack Query + Axios
- Forms/Validation: React Hook Form + Zod
- Auth: JWT (httpOnly cookie khuyến nghị) + refresh token
- Charts/Dashboard: Recharts
- Build: Vite
- E2E (tùy chọn): Playwright

AI/Chatbot (tùy chọn)

- chatbot-service: FastAPI (Python) hoặc Spring Boot adapter
- LLM provider: mock để demo; về sau tích hợp nhà cung cấp phù hợp

- Vector/RAG (nếu cần): pgvector (Postgres) hoặc vector store khác
- Guardrails: permission-aware retrieval + masking dữ liệu nhạy cảm

5. Best practices quan trọng

5.1 Chống chồng chéo lịch (booking conflict)

- Chuẩn hóa slot (30/60 phút) để kiểm tra overlap đơn giản.
- DB constraint/locking để đảm bảo 1 sân không có 2 booking overlap.
- State machine booking: DRAFT → CONFIRMED → IN_PROGRESS → COMPLETED/CANCELED.

5.2 Bảng giá linh động và tính tiền minh bạch

- Tách pricing rules khỏi booking (rule engine nhỏ).
- Lưu snapshot giá tại thời điểm confirm booking.
- Mọi phụ thu/dịch vụ là invoice line riêng, truy vết được.

5.3 Idempotency cho payment và confirm booking

- Confirm booking dùng idempotency key (confirmRequestId).
- Payment dùng paymentRequestId (callback lặp vẫn an toàn).

5.4 Notification và scheduler

- Không gửi trực tiếp trong transaction; ghi Notification record rồi worker gửi.
- Scheduler: nhắc lịch trước giờ chơi; xử lý no-show; đóng phiên khi hết giờ.

5.5 Audit log và phân quyền

- Audit: tạo/đổi/hủy booking, chỉnh giá, payment/refund.
- STAFF giới hạn theo chi nhánh.
- Đối soát doanh thu booking vs doanh thu bán dịch vụ.

5.6 Chatbot AI an toàn

- Bot đọc dữ liệu qua service layer có kiểm quyền.
- Bot chỉ tạo booking draft, người dùng xác nhận.
- Mask dữ liệu nhạy cảm; log intent và latency.

6. Danh sách API tối thiểu (gợi ý)

6.1 Auth

- POST /api/auth/register
- POST /api/auth/login

- GET /api/auth/me

6.2 Venue/Court

- POST /api/branches
- POST /api/courts (type: FOOTBALL/BADMINTON/PICKLEBALL)
- GET /api/courts?branchId=&type=
- GET /api/availability?date=YYYY-MM-DD&branchId=&type=

6.3 Booking

- POST /api/bookings (create draft)
- POST /api/bookings/{id}/confirm (idempotent)
- POST /api/bookings/{id}/cancel
- POST /api/bookings/{id}/reschedule
- GET /api/bookings?from=&to=&branchId=&courtId=&status=

6.4 Pricing

- POST /api/pricing-rules
- GET /api/pricing/quote?courtId=&start=&end=&customerTier=

6.5 Sales (dịch vụ/sản phẩm)

- POST /api/products
- POST /api/orders (attach bookingId hoặc bán lẻ)
- GET /api/orders?date=&branchId=

6.6 Payment

- POST /api/payments (deposit/full, paymentRequestId)
- POST /api/payments/webhook (mock)
- GET /api/receipts?bookingId=

6.7 Reporting

- GET /api/reports/occupancy?from=&to=&branchId=
- GET /api/reports/revenue?from=&to=&branchId=
- GET /api/reports/best-hours?from=&to=&branchId=

6.8 Chatbot

- POST /api/chatbot/messages
- POST /api/chatbot/booking-draft